

MADAPORC

Mini soutenance S4 (2025-2025)

Système Intelligent de Suivi, de Traçabilité et d'Aide à la Décision Reproductive pour l'Élevage Porcin

Présenté par :

Groupe 16-A

Encadreur : Baovola RAZAFINJOELINA

Membres du jury:

- Damo RAZAKARIVELO
- Tovohera ANDRIAMBELOMA

Les membres du groupe 16-A:

ANDRIAMARONDRAIBE Sambatra Mickaella (chef du projet)	ETU004346
RAMAROMANANA Tsanta Mahery Finaritra R. (chef interime)	ETU004590
RAJOELISON Andriamaholy Aina Manoa	ETU004079
ANDRIANANTENAINA Noah	ETU004326
ANDRIAMIADANARIVO Fanilo Tsitohaina	ETU004043
ANDRIANTSIFERANA Nomena Sarobidy	ETU004349
RANOMENJANAHARY Mandresy Ally	ETU004342
RAJAOSON Ny Andry Miaro	ETU004176
RAKOTOARIMANANA Davida Fiderana	ETU004037

SOMMAIRE

PARTIE 1: CADRE DU PROJET ET BESOINS.....	3
1) Présentation du projet MADAPORC:.....	3
1.1) Présentation.....	3
1.2) Contexte Dans de nombreuses exploitations porcines, les informations concernant les animaux sont encore.....	3
1.3) Problématique.....	3
1.4) Solution.....	4
2) Cahier des charges fonctionnel - Madaporc.....	4
2.1) Sécurité.....	4
2.2) Gestion des lots.....	4
2.3) Gestion des pesées.....	5
2.4) Gestion de la reproduction.....	5
2.5) Analyse.....	5
2.6) Gestion sanitaire.....	5
2.7) Gestion des stocks.....	5
2.8) Gestion commerciale et financière.....	6
2.9) Tableau de bord.....	6
2.10) Import / Export.....	6
PARTIE 2: CHOIX TECHNIQUE ET ARCHITECTURE.....	7
1) Introduction.....	7
2) Choix technologique justifié.....	7
2.1) Backend.....	7
2.2) Base de données.....	8
3) Comparaison des technologies – Projet Madaporc.....	8
3.1) Backend : Spring Boot vs Laravel.....	8
3.2) Java 17 vs PHP.....	9
3.3) PostgreSQL vs Oracle Database.....	10
3.4) JSP vs Thymeleaf.....	10
4) Architecture globale du projet.....	11
4.1) Couche de présentation (Views):.....	11
4.2) Couche Contrôleur (Controller):.....	11
4.3) Couche Service:.....	11
4.4) Couche Model:.....	11
4.5) Couche DTO:.....	11
5) WorkFlow du projet:.....	12
Déroulement de l'analyse:.....	12
Cette architecture présente plusieurs avantages :.....	14
6) Architecture technique du projet:.....	14
Autres technologies utilitaires.....	14
PARTIE 3: CONCEPTION DE LA BASE DE DONNÉES.....	15
1) Module Gestion des users:.....	15
2) Module Gestion des porcs:.....	15
3) Module reproduction:.....	16
4) Module Analyse de reproduction:.....	16
5) Module sanitaire:.....	17
6) Module vaccinations:.....	17
7) Module Stock Alimentaire:.....	18
8) Module commercial (clients / ventes):.....	18
9) Module dépenses:.....	18
10) Module Alertes:.....	19
11) Module Import-Export:.....	19
PARTIE 4: PILOTAGE DU PROJET ET PERSPECTIVES.....	20
1) Organisation du projet:.....	20
1.1) Planning:.....	20
1.2) Budget:.....	26
2) Perspectives du projet:.....	28
. Évolutivité technologique (Scalabilité).....	28
. Intelligence Artificielle et Aide à la Décision.....	28
. Intégration Écosystémique.....	28
PARTIE 5: DEMO.....	29
1) Tableau de bord:.....	29
2) Reproduction : groupe et alerte de mise bas.....	29
3) Alerte de reproduction + email:.....	30
4) Détail d'un lot et suivi d'un pesée:.....	31
5) Vente:.....	32
6) Rapport et suivi financier:.....	33
Conclusion:.....	34

PARTIE 1: CADRE DU PROJET ET BESOINS

1) Présentation du projet MADAPORC:

1.1) Présentation

MADAPORC est une application web destinée à la gestion moderne d'une exploitation porcine. Contrairement aux systèmes de gestion classiques qui se limitent à enregistrer des informations administratives, MADAPORC intègre un module d'analyse reproductive permettant d'assister l'éleveur dans ses prises de décision concernant les reproducteurs. Le système assure la traçabilité complète des lots, le suivi sanitaire du cheptel, le suivi des performances de croissance et l'analyse des capacités reproductives des truies et des verrats.

L'objectif principal est d'améliorer la productivité de l'exploitation tout en réduisant les risques liés à une mauvaise gestion du cycle reproductif.

1.2) Contexte

Dans de nombreuses exploitations porcines, les informations concernant les animaux sont encore gérées manuellement ou à travers plusieurs fichiers indépendants.

Cette situation entraîne :

- une perte d'informations historiques
- un manque de traçabilité
- des difficultés dans le suivi des reproducteurs
- une mauvaise anticipation des réformes
- une baisse potentielle de la rentabilité

MADAPORC propose une solution centralisée permettant non seulement de stocker les informations mais également de les analyser afin d'aider l'éleveur à prendre des décisions pertinentes.

1.3) Problématique

Comment mettre en place un système capable de :

- suivre l'évolution complète d'un lot de porcs
- assurer la traçabilité des opérations réalisées
- suivre les performances sanitaires et productives
- analyser automatiquement les performances reproductives

- identifier les reproducteurs performants
- détecter les reproducteurs à réformer
- fournir des indicateurs fiables d'aide à la décision ?

1.4) Solution

Développer MADAPORC, une plateforme web centralisée et intelligente dédiée à la gestion et au pilotage moderne d'une exploitation porcine. La solution s'articule autour de quatre axes majeurs :

- **Suivi et traçabilité**
Enregistrement de l'historique de chaque animal, et traçabilité complète des mouvements de lots.
- **Analyse reproductive assistée**
Automatisation du suivi des saillies, des gestations et des mises bas, avec des algorithmes d'aide à la décision permettant d'identifier les meilleurs reproducteurs et de planifier les réformes.
- **Gestion sanitaire et suivi des performances**
Gestion de traçabilité des vaccinations et suivi des courbes de croissance à partir des différentes pesées.
- **Gestion des flux**
Suivi de la consommation des aliments, contrôle de l'état des stocks d'intrants et calcul de la rentabilité de l'exploitation.

2) Cahier des charges fonctionnel - Madaporc

2.1) Sécurité

Rôles concernés :	• Administrateur • Employé
Règles de gestion :	• Chaque utilisateur possède un identifiant unique. • Seul l'administrateur gère les utilisateurs. • Authentification obligatoire.
Critères de validation :	• Nom d'utilisateur unique. • Mot de passe obligatoire. • Rôle ADMIN ou EMPLOYE.

2.2) Gestion des lots

Rôles concernés :	• Administrateur • Employé
Règles de gestion :	• Chaque lot possède un code unique. • Un lot contient des animaux de même sexe. • Effectif mis à jour automatiquement.
Critères de validation :	• Code obligatoire. • Effectif > 0. • Race et sexe obligatoires.

2.3) Gestion des pesées

Rôles concernés :	• Administrateur • Employé
Règles de gestion :	• Chaque pesée est liée à un lot. • Le gain de poids est calculé automatiquement.
Critères de validation :	• Lot obligatoire. • Poids > 0. • Date obligatoire.

2.4) Gestion de la reproduction

Rôles concernés :	• Administrateur • Employé
Règles de gestion :	• Association d'un lot femelle et d'un lot mâle. • Calcul automatique de la date prévue de mise bas. • Respect de la cohérence des effectifs et des statuts.
Critères de validation :	• Date de saillie obligatoire. • Femelles > 0. • Mâles ≥ 1. • Durée de gestation valide.

2.5) Analyse

Rôles concernés :	• Administrateur • Employé
Règles de gestion :	• Calcul automatique des indicateurs. • Génération automatique de la décision. • Historisation des analyses.
Critères de validation :	• Lot existant. • Taux entre 0 et 100 %.

2.6) Gestion sanitaire

Rôles concernés :	• Administrateur • Employé
Règles de gestion :	• Historique des vaccinations. • Suivi des rappels.
Critères de validation :	• Vaccin, lot et date obligatoires.

2.7) Gestion des stocks

Rôles concernés :	• Administrateur • Employé
Règles de gestion :	• Types : ENTREE, SORTIE, AJUSTEMENT. • Sortie autorisée uniquement si le stock est suffisant. • Mise à jour automatique du stock.
Critères de validation :	• Ingrédient obligatoire. • Quantité > 0. • Date obligatoire.

2.8) Gestion commerciale et financière

Rôles concernés :	• Administrateur
Règles de gestion :	• Chaque vente est liée à un client. • Les dépenses sont catégorisées.
Critères de validation :	• Client obligatoire. • Montant > 0.

2.9) Tableau de bord

Rôles concernés :	• Administrateur • Employé
Règles de gestion :	• Calcul automatique des indicateurs. • Alertes selon les seuils.
Critères de validation :	• Données cohérentes.

2.10) Import / Export

Rôles concernés :	• Administrateur
Règles de gestion :	• Import des formats autorisés. • Validation avant import.
Critères de validation :	• Fichier valide et non vide.

PARTIE 2: CHOIX TECHNIQUE ET ARCHITECTURE

1) Introduction

Pour ce projet, la stack retenue est **Java + Spring Boot** côté backend et **PostgreSQL** pour la base de données. Ce choix n'est pas arbitraire : c'est aujourd'hui l'une des combinaisons les plus solides, les plus éprouvées et les plus utilisées en entreprise pour construire une application robuste et durable. Voici pourquoi, argument par argument, avec les comparaisons face aux principales alternatives.

2) Choix technologique justifié

2.1) Backend

TECHNOLOGIE	ARGUMENTS
	• Le typage statique élimine les erreurs à la source. Le compilateur bloque tout code incohérent avant même l'exécution, alors qu'un langage dynamique comme Python ou JavaScript peut laisser passer ces erreurs jusqu'en production. • La JVM garantit une performance stable sous charge. Ce n'est pas le langage le plus rapide à écrire, mais c'est l'un des plus fiables une fois en exécution, grâce à des décennies d'optimisation. • Java est le standard des systèmes critiques (banque, assurance, grands SI), ce qui n'est pas un hasard : c'est un choix de fiabilité, pas de mode. • Version : OPENJDK: 21.0.11

TECHNOLOGIE	ARGUMENTS
	• L'injection de dépendances est le vrai différenciateur. Elle permet de découpler le code et de tester chaque composant isolément. Express.js ne l'offre pas nativement : il faut tout gérer à la main ou ajouter des librairies tierces non standardisées. • L'écosystème Spring est unifié, alors que Node.js oblige souvent à choisir entre plusieurs librairies concurrentes de qualité inégale pour une même fonctionnalité (authentification, ORM, validation...). • Spring Boot est "production-ready" dès le départ : monitoring (Actuator), sécurité (Spring Security), accès aux données (Spring Data JPA) sont pensés pour fonctionner ensemble sans configuration lourde. • Spring-boot Version : 3.3.5

2.2) Base de données

TECHNOLOGIE	ARGUMENTS
	• Une technologie simple et bien maîtrisée : JSP est une technologie simple à prendre en main . Elle est "basée sur HTML", ce qui la rend accessible. Dans une architecture MVC, JSP joue parfaitement son rôle de couche de présentation, en se concentrant sur l'affichage des données que les contrôleurs lui envoient. • Cohérence dans l'écosystème Java EE : JSP est une technologie standard de l'écosystème Java/Jakarta EE, dans lequel Spring Boot s'inscrit. Pour des développeurs formés à cet environnement, c'est un environnement familier, ce qui facilite la maintenance .

3) Comparaison des technologies – Projet Madaporc

3.1) Backend : Spring Boot vs Laravel

Critère	Argument	Avantage
Gestion des modules métier	Architecture en couches adaptée aux modules Lots, Reproduction, Analyse et Stock.	Spring Boot
Calculs métier	Centralisation des calculs (fertilité, aptitude, recommandations, stock) dans les Services.	Spring Boot
Relations de données	Spring Data JPA simplifie les relations entre les entités.	Spring Boot
Évolutivité	Ajout de nouveaux modules sans impacter les existants.	Spring Boot
Déploiement	Application JAR autonome avec Tomcat embarqué.	Spring Boot

3.2) Java 17 vs PHP

Critère	Argument	Avantage
Fiabilité des calculs	Utilisation de BigDecimal pour les taux et indicateurs.	Java 17
Typage	Enums et LocalDate réduisent les erreurs métier.	Java 17
Organisation	Séparation DTO, Services, Repositories et Models.	Java 17
Maintenance	Les règles métier évoluent sans modifier les vues.	Java 17

3.3) PostgreSQL vs Oracle Database

Critère	Argument	Avantage
Contraintes métier	Support des CHECK, FOREIGN KEY, UNIQUE et colonnes calculées.	Égalité
Volume de données	Adapté aux besoins d'un élevage.	PostgreSQL
Coût	Gratuit et sans licence.	PostgreSQL
Intégration	Excellente intégration avec Spring Data JPA.	Égalité

3.4) JSP vs Thymeleaf

Critère	Argument	Avantage
Compatibilité	Les interfaces sont déjà développées en JSP.	JSP
Affichage	JSTL et EL couvrent les besoins du projet.	Égalité
Maintenance	Correcte si la logique reste côté Service.	JSP
Évolution	Thymeleaf est recommandé pour les nouveaux projets.	Thymeleaf

4) Architecture globale du projet

4.1) Couche de présentation (Views):

Il s'agit de l'interface user, elle a pour rôle de:

- afficher les données
- récupère les saisies user
- envoie les formulaires

4.2) Couche Contrôleur (Controller):

Les contrôleurs:

- reçoivent les requêtes HTTP
- appellent les services
- transmettent les données vers les Views

4.3) Couche Service:

Cette couche :

- effectue les calculs
- applique les règles métier
- valide les données
- coordonne plusieurs repositories

4.4) Couche Model:

Les modèles représentent les tables. Chaque attribut Java correspond à une colonne SQL.

4.5) Couche DTO:

Les DTO servent :

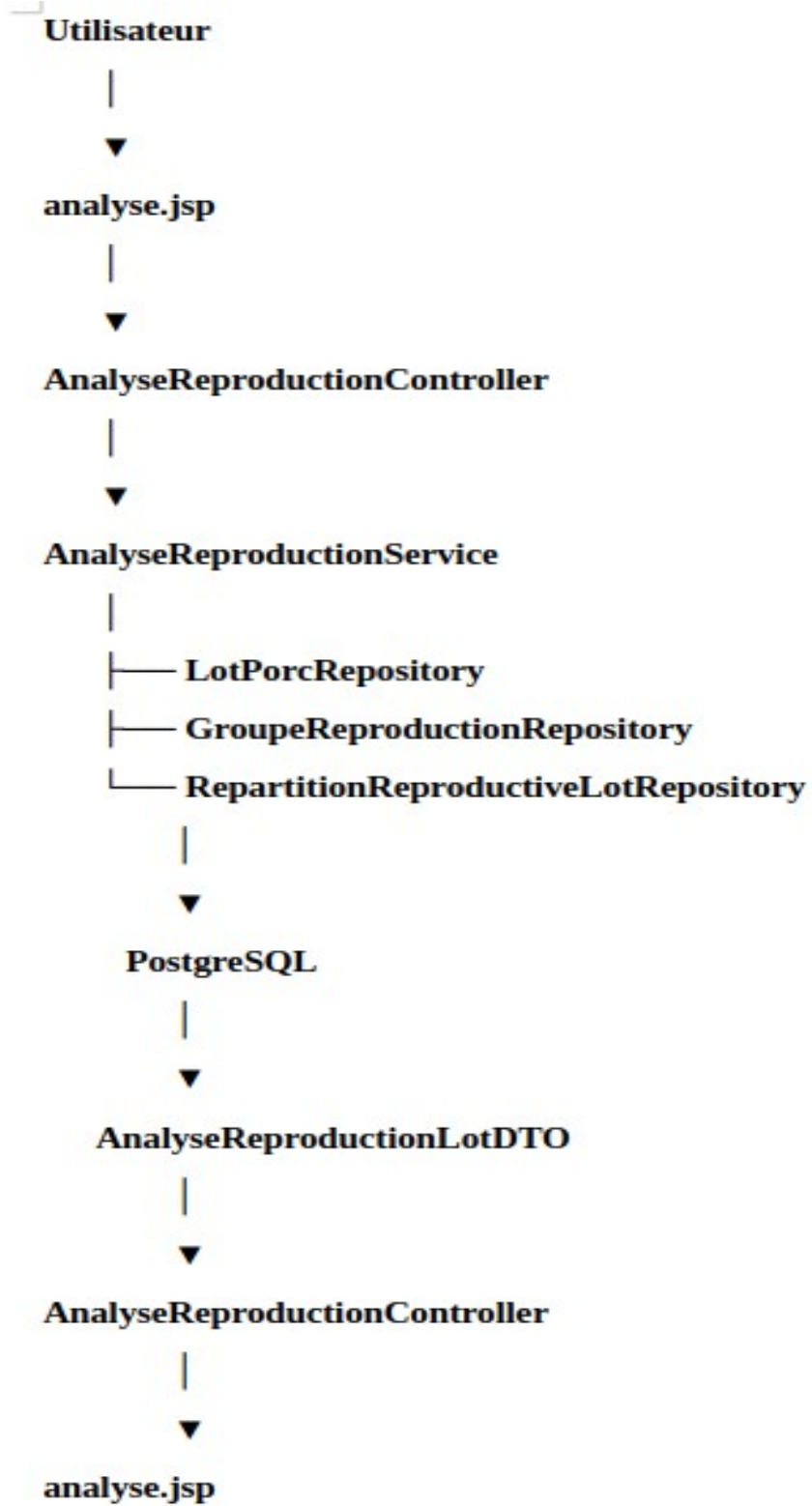
- à transporter des données
- éviter de transmettre directement les entités.

5) WorkFlow du projet:

Prenons comme exemple l'écran Analyse de reproduction:

| Déroulement de l'analyse:

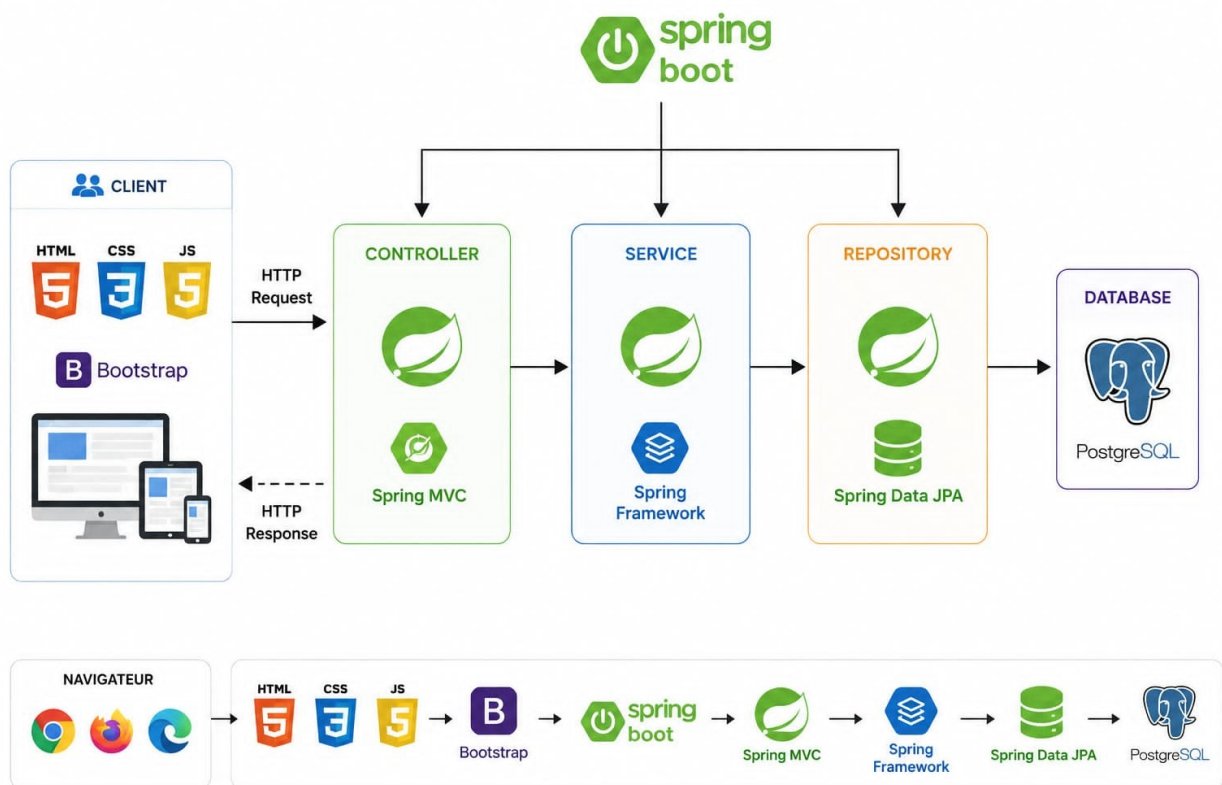
1. L'utilisateur clique sur un lien "Voir analyse" dans son navigateur.
2. Le navigateur envoie une requête HTTP GET vers l'URL correspondante.
3. Spring MVC route la requête vers le contrôleur dédié (ex: ReproductionController).
4. Le contrôleur appelle le service d'analyse (ReproductionService) pour obtenir les indicateurs calculés.
5. Le service effectue les calculs métier, interroge les repositories (LotRepository, ReproductionRepository, etc.) si nécessaire.
6. Les repositories exécutent des requêtes SQL via JPA/Hibernate pour récupérer les données de la base PostgreSQL.
7. Les données remontent : Repository → Service → Contrôleur.
8. Le contrôleur place les données dans le modèle (Model) et renvoie le nom de la vue JSP (ex: "analyse-reproduction").
9. Spring MVC charge la vue JSP correspondante, qui utilise EL et JSTL pour afficher dynamiquement les données.
10. La page HTML est renvoyée au navigateur de l'utilisateur.



Cette architecture présente plusieurs avantages :

- Séparation des responsabilités : chaque couche a un rôle clair.
- Maintenance facilitée : une modification dans une couche impacte peu les autres.
- Réutilisabilité : les services peuvent être appelés par plusieurs contrôleurs.
- **Testabilité** : il est plus simple de tester les services indépendamment des vues et de la base de données.

6) Architecture technique du projet:



Autres technologies utilitaires



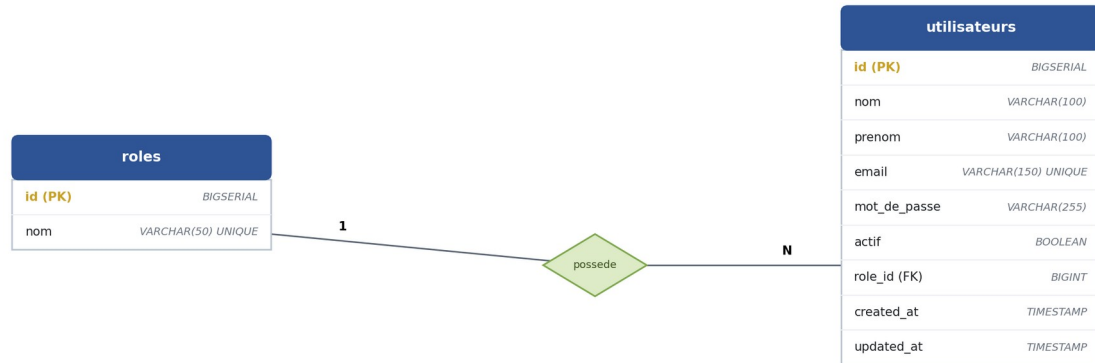
JavaScript



CSS

PARTIE 3: CONCEPTION DE LA BASE DE DONNÉES

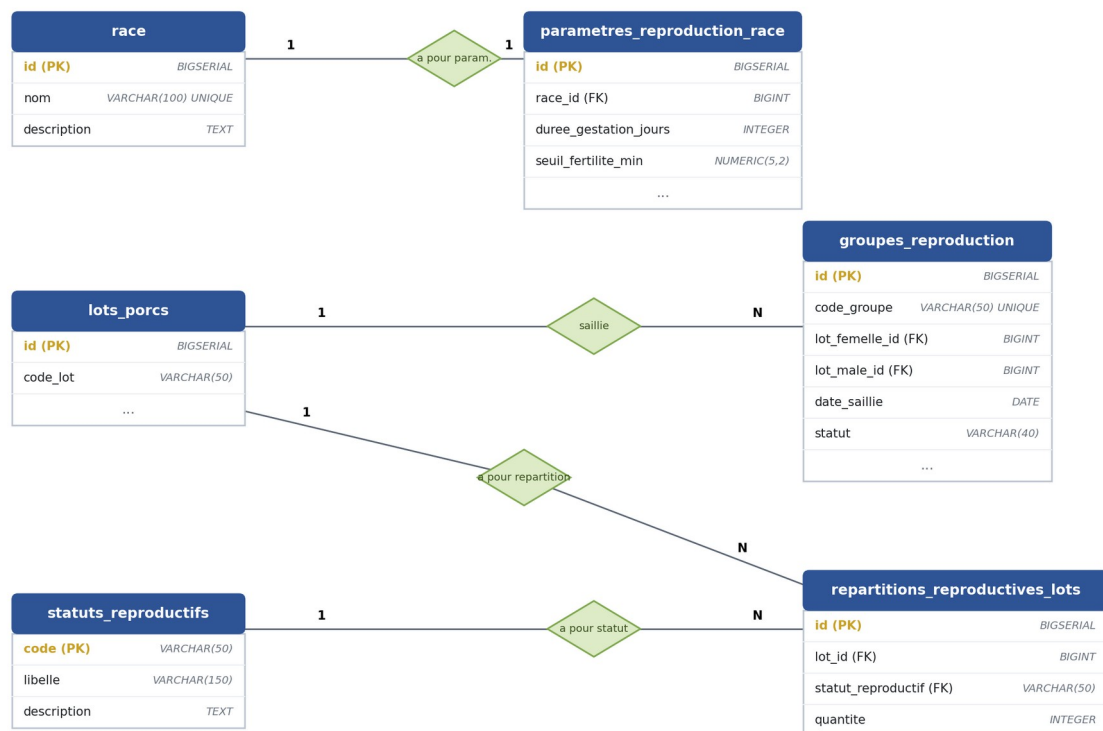
1) Module Gestion des users:



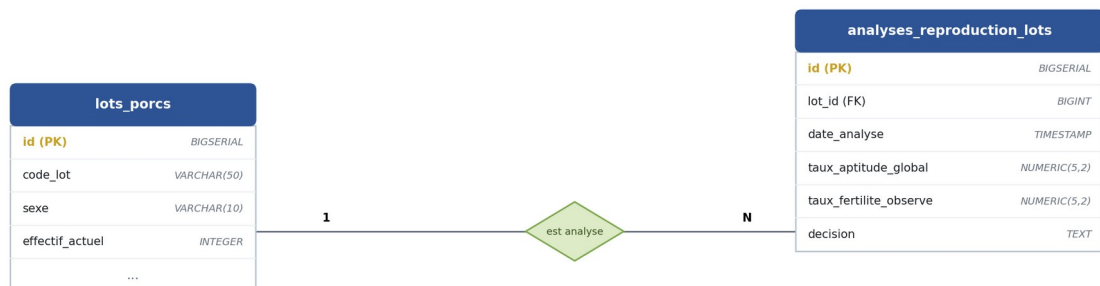
2) Module Gestion des porcs:



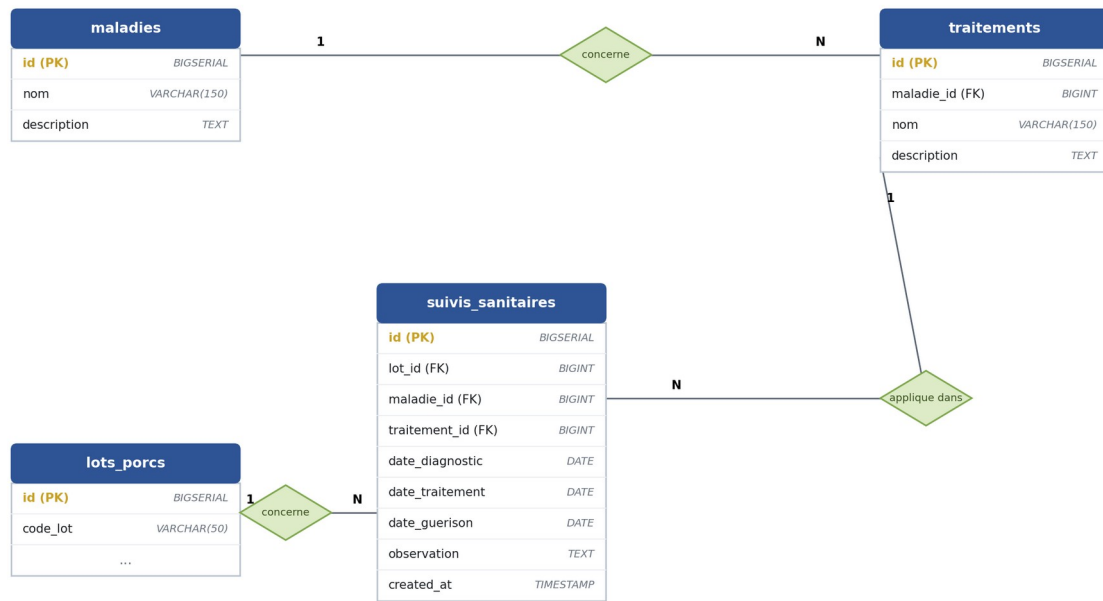
3) Module reproduction:



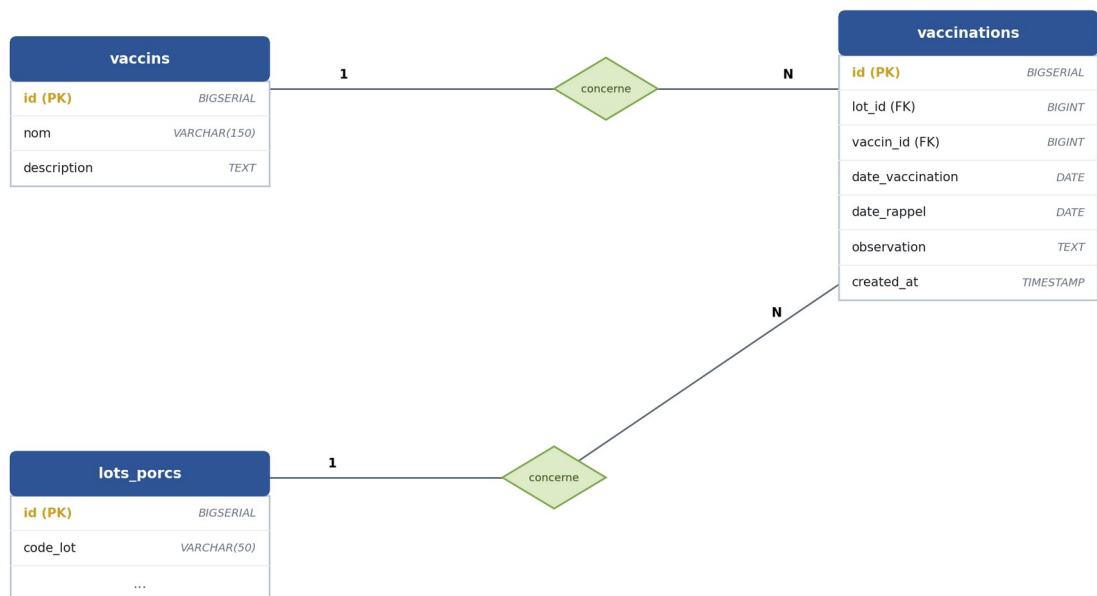
4) Module Analyse de reproduction:



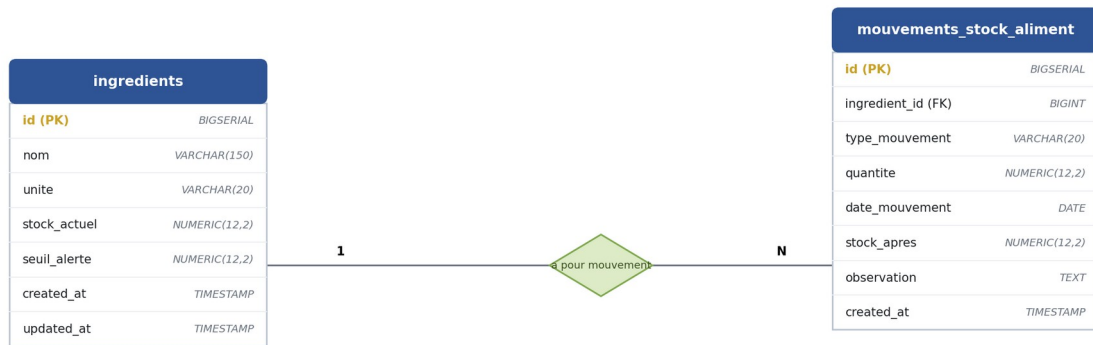
5) Module sanitaire:



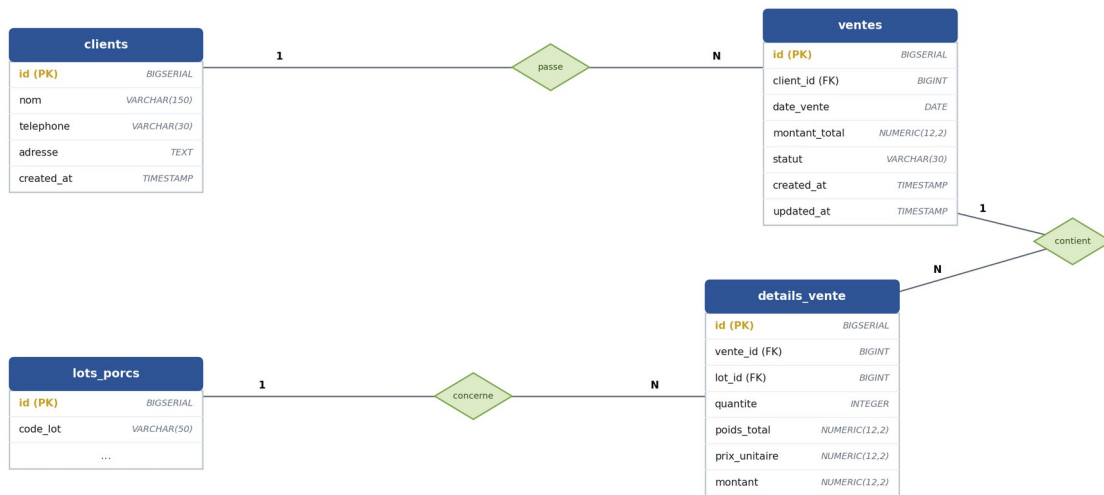
6) Module vaccinations:



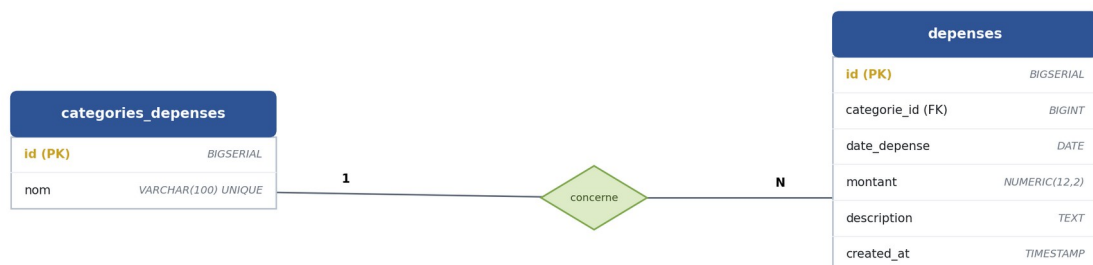
7) Module Stock Alimentaire:



8) Module commercial (clients / ventes):



9) Module dépenses:



10) Module Alertes:



11) Module Import-Export:

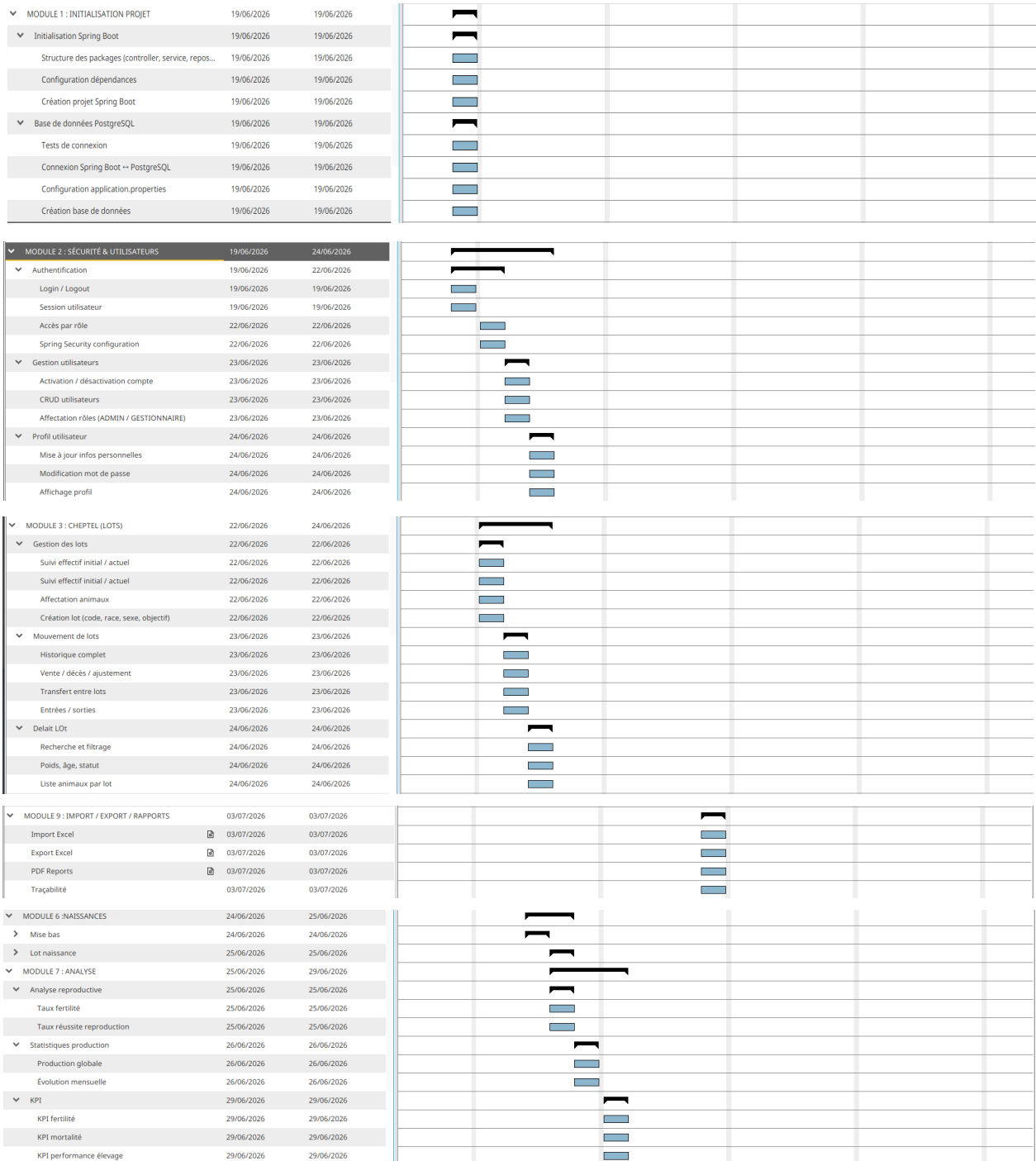


PARTIE 4: PILOTAGE DU PROJET ET PERSPECTIVES

1) Organisation du projet:

1.1) Planning:

1.1.a) Gant



▼ MODULE 4 : SANTÉ	23/06/2026	26/06/2026							
▼ Pesées	23/06/2026	23/06/2026							
Enregistrement poids moyen lot	23/06/2026	23/06/2026							
Historique croissance	23/06/2026	23/06/2026							
Analyse évolution	23/06/2026	23/06/2026							
▼ Vaccins	24/06/2026	24/06/2026							
Gestion référentiel vaccins	24/06/2026	24/06/2026							
Catalogue vaccins	24/06/2026	24/06/2026							
▼ Vaccinations	25/06/2026	25/06/2026							
Planification vaccination	25/06/2026	25/06/2026							
Enregistrement vaccination	25/06/2026	25/06/2026							
Suivi rappels	25/06/2026	25/06/2026							
▼ Suivi sanitaire	26/06/2026	26/06/2026							
Détection maladies	26/06/2026	26/06/2026							
Traitements	26/06/2026	26/06/2026							
Historique médical par lot	26/06/2026	26/06/2026							
▼ MODULE 5 : REPRODUCTION	23/06/2026	26/06/2026							
▼ Gestion reproducteurs	23/06/2026	23/06/2026							
Identification mâles / femelles	23/06/2026	23/06/2026							
Statut reproductif	23/06/2026	23/06/2026							
▼ Groupes de reproduction	24/06/2026	24/06/2026							
Calcul gestation automatique	24/06/2026	24/06/2026							
Saillie planifiée	24/06/2026	24/06/2026							
Création groupe (femelles + mâle)	24/06/2026	24/06/2026							
▼ Cycle reproduction	25/06/2026	25/06/2026							
Mise bas prévue (114 jours)	25/06/2026	25/06/2026							
Statuts cycle	25/06/2026	25/06/2026							
Suivi gestation	25/06/2026	25/06/2026							
› Événements reproduction	26/06/2026	26/06/2026							
▼ MODULE 7 : ANALYSE	25/06/2026	29/06/2026							
▼ Analyse reproductrice	25/06/2026	25/06/2026							
Taux fertilité	25/06/2026	25/06/2026							
Taux réussite reproduction	25/06/2026	25/06/2026							
▼ Statistiques production	26/06/2026	26/06/2026							
Production globale	26/06/2026	26/06/2026							
Évolution mensuelle	26/06/2026	26/06/2026							
▼ KPI	29/06/2026	29/06/2026							
KPI fertilité	29/06/2026	29/06/2026							
KPI mortalité	29/06/2026	29/06/2026							
KPI performance élevage	29/06/2026	29/06/2026							
▼ MODULE 8 : DASHBOARD & ALERTES	30/06/2026	02/07/2026							
▼ Dashboard principal	30/06/2026	30/06/2026							
Affichages obligatoires :	30/06/2026	30/06/2026							
▼ Alertes	01/07/2026	01/07/2026							
Mise bas proche	01/07/2026	01/07/2026							
Retard reproduction	01/07/2026	01/07/2026							
Vaccination urgente	01/07/2026	01/07/2026							
› Tableau global	02/07/2026	02/07/2026							
▼ MODULE 10 : FINALISATION	06/07/2026	07/07/2026							
▼ Intégration	06/07/2026	06/07/2026							
Merge modules	06/07/2026	06/07/2026							
Résolution conflits	06/07/2026	06/07/2026							
Debug	07/07/2026	07/07/2026							
Tests globaux	07/07/2026	07/07/2026							

1.2.a) Excel

Mickaella (chef de projet)

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/ GroupeReproductionController.java	@GetMapping(/reproduction/groupe/{id}) detail(id, Model)	30	Terminé	
2	1ère part	Controller	controller/ GroupeReproductionController.java	@GetMapping(/{id}/confirmer-mise-bas) afficherFormulaireMiseBas(id, Model)	30	Terminé	Err:520
3	1ère part	Controller	controller/ GroupeReproductionController.java	@PostMapping(/{id}/confirmer-mise-bas) confirmerMiseBas(id, dto, Model)	30	Terminé	
4	1ère part	Controller	controller/ GroupeReproductionController.java	@PostMapping(/{id}/cloturer) cloturer(id)	30	Terminé	Implémentée
5	1ère part	Service	service/ GroupeReproductionMiseBasService.java	getDetailGroupe(groupeId)	30	Terminé	
6	1ère part	Service	service/ GroupeReproductionMiseBasService.java	calculerJoursRestants(groupeId)	30	Terminé	
7	1ère part	Service	service/ GroupeReproductionMiseBasService.java	calculerPourcentageEvolution(groupeId)	30	Terminé	
8	1ère part	Service	service/ GroupeReproductionMiseBasService.java	confirmerMiseBas(groupeId, dto)	30	Terminé	
9	1ère part	Service	service/ GroupeReproductionMiseBasService.java	validerDonneesMiseBas(dto)	30	Terminé	Présente
10	1ère part	Service	service/ GroupeReproductionMiseBasService.java	mettreAJourGroupeApresMiseBas(groupeId, dto)	30	Terminé	Présente
11	1ère part	Service	service/ GroupeReproductionMiseBasService.java	mettreAJourRepartitionApresMiseBas(lotId, groupe)	30	Terminé	Via repartitionReproductiveService.mettreAJourApresMiseBas
12	1ère part	Service	service/ GroupeReproductionMiseBasService.java	cloturer(groupeId)	30	Terminé	Implémentée
13	1ère part	Service	service/LotNaissanceService.java	creerLotNaissanceApresMiseBas(groupeId, dto)	30	Terminé	Implémentée (création lot + lien lot mère/groupe)
14	1ère part	Service	service/LotNaissanceService.java	preparerLotNaissanceDepuisGroupe(groupeId)	30	Terminé	Implémentée
15	1ère part	DTO	dto/ConfirmationMiseBasDTO.java · LotNaissanceDTO.java	DTO mise bas + lot naissance	20	Terminé	
16	1ère part	Vue	reproduction/groupe/detail.jsp · confirmerMiseBas.jsp	Détail groupe + barre évolution + confirmation	40	Terminé	
17	Suite	Controller	controller/RapportController.java	@GetMapping(/rapports) index(RapportFiltreDTO, Model)	30	Terminé	Implémenté (bilan financier filtré par dates)
18	Suite	Service	service/RapportService.java	genererRapport · calculerTotalVentes (VALIDEE) · calculerTotalDepenses · calculerBeneficeNet · compterVentes · compterDepenses · validerFiltre	210	Terminé	Implémenté
19	Suite	DTO	dto/RapportFiltreDTO.java	Filtre dates rapport (dateDebut, dateFin)	10	Terminé	Complété
20	Suite	DTO	dto/RapportFinancierDTO.java	totalVentes · totalDepenses · beneficeNet · nbVentes · nbDepenses	10	Terminé	Créé
21	Suite	Vue	rapports/index.jsp	Bilan financier + filtre dates	20	Terminé	Bloc financier ajouté
TOTAL (min)					760		

Tsanta (chef Interime)

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/ GroupeReproductionController.java	@GetMapping(/reproduction/groupe) listGroupe(Model)	30	Terminé	
2	1ère part	Controller	controller/ GroupeReproductionController.java	@GetMapping(/reproduction/groupe/form) showForm(id, Model)	30	Terminé	
3	1ère part	Controller	controller/ GroupeReproductionController.java	@PostMapping(/reproduction/groupe/save) save(dto, Model, HttpSession)	30	Terminé	
4	1ère part	Service	service/ GroupeReproductionCreationService.java	creer(dto, utilisateurId)	30	Terminé	
5	1ère part	Service	service/ GroupeReproductionCreationService.java	modifier(id, dto)	30	Terminé	
6	1ère part	Service	service/ GroupeReproductionCreationService.java	calculerDatePrevueMiseBas(dateSaillie, dureeGestation)	30	Terminé	
7	1ère part	Service	service/ GroupeReproductionCreationService.java	verifierLotFemelle · verifierLotMale · verifierFemellesDisponibles	90	Terminé	
8	1ère part	Service	service/ GroupeReproductionCreationService.java	mettreAJourRepartitionApresSaillie(lotId, nombreFemelles)	30	Terminé	
9	1ère part	Service	service/ GroupeReproductionQueryService.java	findAll() · getDetailGroupe(id) · prepareFormModel(Model, id)	90	Terminé	
10	1ère part	Repository	repository/ GroupeReproductionRepository.java	findAllByOrderByIdDateSaillieDesc · findByLotFemelleId · countByStatutIn	45	Terminé	
11	1ère part	Repository	repository/ ParametreReproductionRaceRepository.java	findByRaceId(raceId)	15	Terminé	
12	1ère part	Model/DTO/ Vue	model/GroupeReproduction.java · ParametreReproductionRace.java · dto/GroupeReproductionDTO.java · GroupeReproductionDetailDTO.java · groupe/liste.jsp · form.jsp	Entités + DTO + vues création	90	Terminé	
13	Suite	Controller	controller/ImportExportController.java	indexImports · showImportForm · importerExcel · exporterExcel · exporterPdf · historique	180	Terminé	
14	Suite	Service	service/ImportExportService.java	importerLots/Clients/Vaccinations/Ingredients Excel · exporterLots/Ventes/Groupe/Analyse Excel · tracerImport · tracerExport · historique	150	Terminé	
15	Suite	Repository	repository/ImportExportRepository.java	byModule · byStatut · orderByDate	45	Terminé	
16	Suite	Model	model/ImportExport.java	Entité (IMPORT/EXPORT, EXCEL/PDF, SUCCES/ECHEC)	15	Terminé	
17	Suite	DTO	dto/ImportExcelDTO.java · RapportFiltreDTO.java	DTO import	20	Terminé	
18	Suite	Vue	imports/form.jsp · imports/historique.jsp	Vues import + historique	40	Terminé	
TOTAL (min)					990		

Manoa

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/AnalyseReproductionController.java	@GetMapping(/reproduction/analyse/lots/{lotId}) analyseLot(lotId, Model)	30	Terminé	
2	1ère part	Controller	controller/AnalyseReproductionController.java	@PostMapping(/reproduction/analyse/generer/{lotId}) genererAnalyse(lotId, Model)	30	Terminé	
3	1ère part	Service	service/AnalyseReproductionService.java	analyserLot(lotId)	30	Terminé	
4	1ère part	Service	service/AnalyseReproductionService.java	calculerTauxAptitudeGlobal(dto) · calculerTauxRecommande(dto) · calculerTauxFertiliteObserve(dto)	90	Terminé	
5	1ère part	Service	service/AnalyseReproductionService.java	genererDecision(dto) · enregistrerAnalyse(dto)	60	Terminé	
6	1ère part	Service	service/RepartitionReproductiveService.java	initialiserRepartitionLotFemelle(lotId) · getRepartitionByLot(lotId)	60	Terminé	
7	1ère part	Service	service/RepartitionReproductiveService.java	calculerFemellesDisponibles(lotId)	30	Terminé	Terminé (refactorisé) : logique dans initialiserRepartitionLotFemelle /
8	1ère part	Service	service/RepartitionReproductiveService.java	calculerFemellesEnCycle(lotId)	30	Terminé	Terminé (refactorisé) : getQuantite(lotId, "EN_CYCLE")
9	1ère part	Service	service/RepartitionReproductiveService.java	mettreAJourRepartition(lotId, statut, quantite)	30	Terminé	
10	1ère part	Repository	repository/AnalyseReproductionLotRepository.java	findByLotIdOrderByDateAnalyseDesc(lotId)	15	Terminé	
11	1ère part	Repository	repository/RepartitionReproductiveLotRepository.java	findByLotId · findByLotIdAndStatutReproductif	30	Terminé	
12	1ère part	Repository	repository/StatutReproductifRepository.java	findAll()	15	Terminé	
13	1ère part	Model/DTO/ Vue	model/AnalyseReproductionLot.java · RepartitionReproductiveLot.java · StatutReproductif.java · dto/AnalyseReproductionLotDTO.java · RepartitionReproductiveDTO.java · analyseLot.jsp	Entités + DTO + vue analyse	90	Terminé	
14	Suite	Controller	controller/MouvementStockController.java	listMouvements · showForm · saveMouvement(ingredientId, typeMouvement, quantite, Model)	90	Terminé	Présent (46 lignes)
15	Suite	Service	service/MouvementStockService.java	enregistrerMouvementStock · verifierStockDisponible	60	Terminé	Présent (101 lignes)
16	Suite	Service	service/MouvementStockService.java	appliquerEntree · appliquerSortie	60	Terminé	Présent : appliquerEntree (L71) /
17	Suite	Repository	repository/MouvementStockAlimentRepository.java	orderByDate · byIngredient	30	Terminé	Renommé : MouvementStockRepository.findAllBy
18	Suite	Model	model/MouvementStockAliment.java	Entité (ENTREE/SORTIE/AJUSTEMENT)	15	Terminé	Renommé :
19	Suite	DTO/ Vue	dto/MouvementStockDTO.java · stocks/mouvements.jsp · formMouvement.jsp	DTO + vues mouvements stock	45	Terminé	
TOTAL (min)					840		

Noah

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/DashboardController.java	@GetMapping(/dashboard) dashboard(Model, HttpSession)	30	Terminé	
2	1ère part	Service	service/DashboardService.java	getDashboard(debut, fin)	30	Terminé	
3	1ère part	Service	service/DashboardService.java	compterLotsActifs · calculerTotalPorcsActifs · compterGroupesReproductionActifs · compterMisesBasProches	120	Terminé	
4	1ère part	Service	service/DashboardService.java	calculerTauxAptitudeGlobal · calculerTauxFertiliteGlobal	60	Terminé	
5	1ère part	Service	service/DashboardService.java	calculerVentresMois · calculerDepensesMois · calculerBeneficeNet	90	Terminé	
6	1ère part	Service	service/DashboardService.java	listerStocksFaibles · listerVaccinationsAVenir(dateLimite)	60	Terminé	
7	1ère part	DTO	dto/DashboardDTO.java	DTO indicateurs	10	Terminé	
8	1ère part	Vue	dashboard/index.jsp	Tableau de bord + graphes	20	Terminé	Graphe d'évolution ne montre pas montées/chutes + bouton générer rapport KO (correction.md)
9	1ère part	Controller	controller/AlerteReproductionController.java	@GetMapping(/reproduction/alertes) listAlertes(Model)	30	Terminé	
10	1ère part	Controller	controller/AlerteReproductionController.java	@PostMapping(/{id}/lire) marquerCommeLue(id)	30	Terminé	
11	1ère part	Controller	controller/AlerteReproductionController.java	@PostMapping(/{id}/traiter) traiter(id)	30	Terminé	
12	1ère part	Service	service/AlerteReproductionService.java	listerAlertesActives · genererAlertesMiseBasProche · marquerCommeLue · traiter	120	Terminé	
13	1ère part	Repository	repository/AlerteReproductionRepository.java	findByStatut · findByTypeAlerteAndStatut	30	Terminé	
14	1ère part	Model/DTO/ Vue	model/AlerteReproductionDTO.java · dto/AlerteReproductionDTO.java · reproduction/alertes/liste.jsp	Entité + DTO + vue alertes	45	Terminé	
15	Suite	Controller	controller/IngredientController.java	listIngredients · showForm · save	90	Terminé	
16	Suite	Service	service/IngredientService.java	findAllIngredients · creerIngredient · modifierIngredient · listerStocksFaibles	120	Terminé	
17	Suite	Repository	repository/IngredientRepository.java	findAllByOrderByNomAsc · existsByNomIgnoreCase · findStocksFaibles	45	Terminé	
18	Suite	Model/DTO/ Vue	model/IngredientDTO.java · dto/IngredientDTO.java · stocks/ingredients.jsp · formIngredient.jsp	Entité + DTO + vues ingrédients	60	Terminé	
TOTAL (min)					1020		

Mandresy

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut
0	All Interface	All JSP			200	Terminé
1	Suite	Model	model/Depense.java	Entité Dépense (categorie, dateDepense, montant, description)	15	Terminé
2	Suite	Model	model/CategorieDepense.java	Entité CategorieDepense (id, nom)	15	Terminé
3	Suite	DTO	dto/DepenseDTO.java	DTO Dépense	10	Terminé
4	Suite	DTO	dto/CategorieDepenseDTO.java	DTO Catégorie	10	Terminé
5	Suite	Repository	repository/CategorieDepenseRepository.java	findAllByOrderByNameAsc · existsByNomIgnoreCase	30	Terminé
6	Suite	Service	service/CategorieDepenseService.java	findAllCategories · findById · creer · modifier · valider	150	Terminé
7	Suite	BD	resources/data.sql	Catégories (Alimentation, Santé, Personnel, Transport, Maintenance, Autre)	25	Terminé
8	Renfort	Santé	sante/* · layout/sidebar.jsp	Renfort module Santé (vaccin/vaccination/traitement/maladie) + sidebar	40	Terminé
TOTAL (min)					495	

Fanilo

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut
1	1ère part	Controller	controller/LotPorcController.java	@GetMapping(/lots) listLots(LotFiltreDTO, Model)	30	Terminé
2	1ère part	Controller	controller/LotPorcController.java	@GetMapping(/lots/form) showForm(id, Model)	30	Terminé
3	1ère part	Controller	controller/LotPorcController.java	@PostMapping(/lots/save) save(LotPorcDTO, Model)	30	Terminé
4	1ère part	Controller	controller/LotPorcController.java	@GetMapping(/lots/{id}) detail(id, Model)	30	Terminé
5	1ère part	Controller	controller/LotPorcController.java	@PostMapping(/lots/archive/{id}) archiver(id)	30	Terminé
6	1ère part	Service	service/LotPorcService.java	rechercherLots(filtre) · prepareFormModel · creerLot · modifierLot · getDetailLot · archiverLot · existeCodeLot · verifierLotActif	240	Terminé
7	1ère part	Repository	repository/LotPorcRepository.java	existsByCodeLot · findByCodeLotContainingIgnoreCase · findBySexe · findByObjectif · findByStatut · countByStatut	90	Terminé
8	1ère part	Model/DTO	model/LotPorc.java · Race.java · dto/LotPorcDTO.java · LotFiltreDTO.java · LotDetailDTO.java	Entités + DTO lots	75	Terminé
9	1ère part	Vue	lots/listeLots.jsp · formLot.jsp · detailLot.jsp	Vues lots	60	Terminé
10	1ère part	Controller	controller/MouvementLotController.java	@GetMapping(/lots/{id}/mouvements) listMouvements(id, Model)	30	Terminé
11	1ère part	Controller	controller/MouvementLotController.java	@PostMapping(/lots/mouvements/save) saveMouvement(MouvementLotDTO, Model)	30	Terminé
12	1ère part	Service	service/MouvementLotService.java	getMouvementsByLot · enregistrerMouvement · augmenterEffectif · diminuerEffectif · verifierQuantiteDisponible	150	Terminé
13	1ère part	Repository	repository/MouvementLotPorcRepository.java	findByLotIdOrderByDateMouvementDesc(lotId)	15	Terminé
14	1ère part	Model/DTO/ Vue	model/MouvementLotPorc.java · dto/MouvementLotDTO.java · lots/mouvements.jsp	Mouvements d'effectif	45	Terminé
15	Suite	Controller	controller/VenteController.java	listVentes · showForm · save · valider(/ventes/valider/{id}) · annuler · genererRecuPdf	180	Terminé
16	Suite	Service	service/VenteService.java	findAll · creerVente · validerVente · annulerVente · calculerMontantTotal · genererRecuPdf	180	Terminé
17	Suite	Repository	repository/VenteRepository.java · DetailVenteRepository.java	byDateVente · byStatut · byVenteId	45	Terminé
18	Suite	Model	model/Vente.java · DetailVente.java	Entités (statuts BROUILLON/VALIDEE/ANNULEE)	30	Terminé
19	Suite	DTO	dto/VenteDTO.java · DetailVenteDTO.java	DTO ventes	20	Terminé
20	Suite	Vue	commerce/ventes.jsp · formVente.jsp · detailVente.jsp	Vues ventes	60	Terminé
TOTAL (min)					1400	

Nomena

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut
1	1ère part	Controller	controller/PeseeLotController.java	@GetMapping(/lots/{id}/pesees) listPesees(id, Model)	30	Terminé
2	1ère part	Controller	controller/PeseeLotController.java	@PostMapping(/lots/pesees/save) savePesee(PeseeLotDTO, Model)	30	Terminé
3	1ère part	Service	service/PeseeLotService.java	getPeseesByLot(lotId) · enregistrerPesee(dto) · calculerEvolutionPoids(lotId)	90	Terminé
4	1ère part	Repository	repository/PeseeLotRepository.java	findByLotIdOrderByDatePeseeAsc(lotId)	15	Terminé
5	1ère part	Model/DTO/ Vue	model/PeseeLot.java · dto/PeseeLotDTO.java · lots/pesees.jsp	Pesées	45	Terminé
6	1ère part	Controller	controller/VaccinController.java	/vaccins · /vaccins/form · /vaccins/save	90	Terminé
7	1ère part	Controller	controller/VaccinationController.java	/vaccinations · /vaccinations/form · /vaccinations/save	90	Terminé
8	1ère part	Controller	controller/SuiviSanitaireController.java	/sante/suivis · /sante/suivis/form · /sante/suivis/save	90	Terminé
9	1ère part	Service	service/VaccinService.java · VaccinationService.java · SuiviSanitaireService.java	Logique métier vaccins / vaccinations / suivis	30	Terminé
10	1ère part	Repository	Vaccin · Vaccination · Maladie · Traitement · SuiviSanitaire Repository	Accès données santé	15	Terminé
11	1ère part	Model	Vaccin · Vaccination · Maladie · Traitement · SuiviSanitaire	Entités santé	75	Terminé
12	1ère part	DTO	VaccinDTO · VaccinationDTO · SuiviSanitaireDTO	DTO santé	30	Terminé
13	1ère part	Vue	sante/vaccins · formVaccin · vaccinations · formVaccination · suivis · formSuivi	Vues santé	120	Terminé
TOTAL (min)					750	

Davida

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut
1	1ère part	Controller	controller/AuthController.java	@GetMapping(/login) showLogin(Model)	30	Terminé
2	1ère part	Controller	controller/AuthController.java	@PostMapping(/login) login(LoginDTO, Model, HttpSession)	30	Terminé
3	1ère part	Controller	controller/AuthController.java	@GetMapping(/logout) logout(HttpSession)	30	Terminé
4	1ère part	Controller	controller/UtilisateurController.java	@GetMapping(/utilisateurs) listUtilisateurs(Model)	30	Terminé
5	1ère part	Controller	controller/UtilisateurController.java	@GetMapping(/utilisateurs/form) showForm(id, Model)	30	Terminé
6	1ère part	Controller	controller/UtilisateurController.java	@PostMapping(/utilisateurs/save) save(UtilisateurDTO, Model)	30	Terminé
7	1ère part	Controller	controller/UtilisateurController.java	@PostMapping(/utilisateurs/desactiver/{id}) desactiver(id)	30	Terminé
8	1ère part	Service	service/AuthService.java	connecter(LoginDTO, HttpSession)	30	Terminé
9	1ère part	Service	service/AuthService.java	verifierMotDePasse(brut, hash)	30	Terminé
10	1ère part	Service	service/AuthService.java	deconnecter(HttpSession)	30	Terminé
11	1ère part	Service	service/UtilisateurService.java	findAll() · findById(id) · creer(dto) · modifier(id,dto) · desactiver(id) · emailExiste(email)	180	Terminé
12	1ère part	Repository	repository/UtilisateurRepository.java	findByEmail(email) · existsByEmail(email)	30	Terminé
13	1ère part	Model/DTO	model/Utilisateur.java · Role.java · dto/LoginDTO.java · UtilisateurDTO.java	Entités + DTO	60	Terminé
14	1ère part	Vue	login.jsp · utilisateurs/liste.jsp · utilisateurs/form.jsp	Connexion + liste/form utilisateurs	60	Terminé
15	Suite	Controller	controller/ClientController.java	@GetMapping(/clients) listClients(Model)	30	Terminé
16	Suite	Controller	controller/ClientController.java	@GetMapping(/clients/form) showClientForm(id, Model)	30	Terminé
17	Suite	Controller	controller/ClientController.java	@PostMapping(/clients/save) saveClient(ClientDTO, Model)	30	Terminé
18	Suite	Controller	controller/ClientController.java	@GetMapping(/clients/{id}) detailClient(id, Model)	30	Terminé
19	Suite	Service	service/ClientService.java	findAll · findById · creer · modifier · getForm	150	Terminé
20	Suite	Repository	repository/ClientRepository.java	findAllByOrderByNomAsc · existsByNomIgnoreCase	30	Terminé
21	Suite	Model	model/Client.java	Entité Client	15	Terminé
22	Suite	DTO	dto/ClientDTO.java	DTO Client	10	Terminé
23	Suite	Vue	commerce/clients.jsp · commerce/formClient.jsp	Liste + formulaire clients	40	Terminé
TOTAL (min)					995	

Miario

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut
1	1ère part	Projet	pom.xml	Configuration Maven + dépendances	20	Terminé
2	1ère part	Projet	MadaporcApplication.java	Classe principale Spring Boot	20	Terminé
3	1ère part	Config	application.properties	Datasource PostgreSQL + vues JSP + encodage	15	Terminé
4	1ère part	Common	common/AppConstants.java	Constantes globales	10	Terminé
5	1ère part	BD	resources/schema.sql	Création des tables	25	Terminé
6	1ère part	BD	resources/data.sql	Données de test	25	Terminé
7	1ère part	CSS	static/css/app.css	Feuille de style	20	Terminé
8	1ère part	Vue	layout/header.jsp	En-tête	20	Terminé
9	1ère part	Vue	layout/sidebar.jsp	Menu latéral	20	Terminé
10	1ère part	Vue	layout/footer.jsp	Pied de page	20	Terminé
11	1ère part	Vue	login.jsp · placeholder.jsp	Pages globales	40	Terminé
12	Suite	Controller	controller/DepenseController.java	@GetMapping(/depenses) listDepenses(Model)	30	Terminé
13	Suite	Controller	controller/DepenseController.java	@GetMapping(/depenses/form) showForm(id, Model)	30	Terminé
14	Suite	Controller	controller/DepenseController.java	@PostMapping(/depenses/save) save(DepenseDTO, Model)	30	Terminé
15	Suite	Service	service/StockService.java	findAllDepenses()	30	Terminé
16	Suite	Service	service/StockService.java	enregistrerDepense(DepenseDTO)	30	Terminé
17	Suite	Repository	repository/DepenseRepository.java	findAllByOrderByDateDepenseDesc / between / byCategorie / totalDepenses	15	Terminé
18	Suite	Vue	finance/depenses.jsp · finance/formDepense.jsp	Liste + formulaire dépenses	40	Terminé
TOTAL (min)					440	

La conception en plus de la réalisation du projet a mis 19 jours soit environ 3 semaines.

1.2) Budget:

Membre	Rôle	Temps(min)	Temps(heures)	Tarif horaire	Budget Total
Miario	Dev Junior	440	7.33	88 185	646 396,05
Davida	Dev Junior	995	16.58	88 185	1 462 401,25
Fanilo	Dev Junior	1400	23.33	88 185	2 057 356 ,05
Sarobidy	Dev Junior	750	12.5	88 185	1 102 312,5
Tsanta	Dev Junior	990	16.5	88 185	1 455 052,5
Mickaella	Dev Junior	760	12.67	107 000	1 355 690
Manoa	Dev Junior	840	14	88 185	1 234 590
Noah	Dev Junior	1020	17	88 185	1 499 145

Consommation électrique par membre (exmple Acer Nitro V15, 100 W, tarif JIRAMA 436 Ar/kWh) :

Membre	Temps (min)	Temps (h)	Consommation (kWh)	Coût (Ar)
Miario	440	7,33	0,73	319,73
Davida	995	16,58	1,66	723,03
Fanilo	1400	23,33	2,33	1017,33
Sarobidy	750	12,5	1,25	545
Tsanta	990	16,5	1,65	719,4
Mickaella	760	12,67	1,27	552,27

Manoa	840	14	1,4	610,4
Noah	1020	17	1,7	741,2
Total		119,92 h	11,99 kWh	5 228,37 Ar

Coût employeur (salaire brut + charges patronales à 19 %) :

Membre	Salaire brut (Ar)	Taux patronal	Charges patronales (Ar)	Coût employeur (Ar)
Miara	646 690	19%	122 871,10	769 561,10
Davida	1 462 401,25	19%	277 856,24	1 740 257,49
Fanilo	2 057 650	19%	390 953,50	2 448 603,50
Sandroy	1 102 312,50	19%	209 439,38	1 311 751,88
Tsanta	1 455 052,50	19%	276 459,98	1 731 512,48
Mickaella	1 355 333,33	19%	257 513,33	1 612 846,66
Manoa	1 234 590	19%	234 572,10	1 469 162,10
Noah	1 499 145	19%	284 837,55	1 783 982,55
TOTAL			2 054 503,18	12 867 677,76

Synthèse du budget :

Poste budgétaire	Montant
Salaires bruts	10 813 174,58 Ar
Charges patronales à 19 %	2 054 503,17 Ar
Électricité JIRAMA estimée	6 300,00 Ar
Budget total estimé	12 873 977,75 Ar

2) Perspectives du projet:

Le projet MADAPORC a été conçu pour répondre au besoin critique de structuration de la filière porcine à travers une gestion technique précise et une analyse reproductive automatisée. Si la version actuelle pose les fondations nécessaires à la gestion de cycle, de santé et de traçabilité, elle offre un potentiel d'évolution significatif pour s'adapter aux réalités du terrain et aux avancées technologiques.

. Évolutivité technologique (Scalabilité)

Bien que l'architecture actuelle repose sur une pile robuste (Spring Boot MVC, JPA, PostgreSQL), nous envisageons pour les versions futures :

- Transition vers une architecture RESTful API : Passer progressivement à une architecture découplée, permettant une meilleure réactivité et une séparation nette entre la logique métier et la couche présentation (type React ou Vue.js).
- Optimisation des données : À mesure que le volume d'historisation (pesées, suivis sanitaires) augmentera, nous prévoyons l'implémentation de solutions de mise en cache (Redis) pour garantir la fluidité du dashboard et des rapports générés.

. Intelligence Artificielle et Aide à la Décision

L'analyse reproductive développée constitue le socle d'une future intelligence métier plus poussée :

- Modèles prédictifs : Intégrer des algorithmes de Machine Learning pour prédire les cycles de chaleurs et optimiser le taux de fertilité, en se basant sur l'historique des données collectées (taux de survie des porcelets, performance des races).
- Automatisation des alertes : Transformer le système d'alertes actuel en un moteur de notification intelligent capable d'envoyer des alertes proactives avant même l'apparition d'une anomalie sanitaire détectée via les suivis de poids.

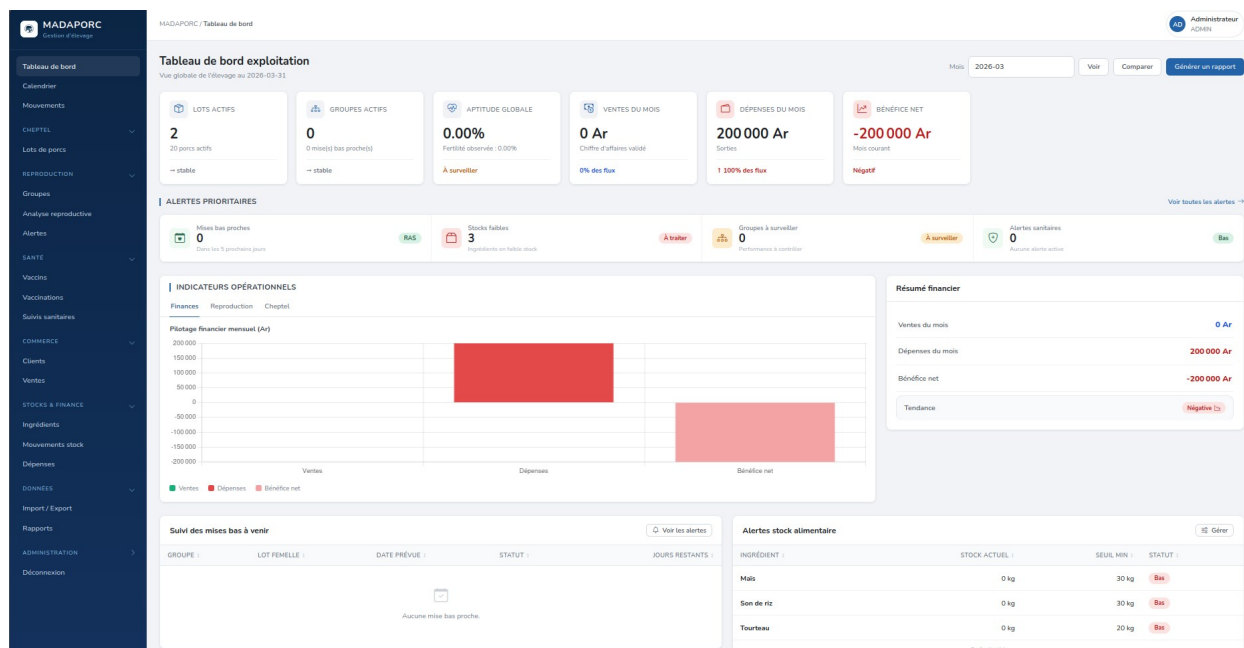
. Intégration Écosystémique

- Couplage GIS : En tirant profit des systèmes d'information géographique, nous pourrions cartographier les fermes sur une plateforme dynamique pour visualiser les flux de cheptel et les zones à risque sanitaire, offrant ainsi une vision spatiale en plus de la gestion statistique.
- Interopérabilité : Faciliter l'échange de données avec d'autres systèmes de gestion agricole (via Webhooks) pour s'intégrer dans un écosystème agricole national plus large, favorisant la traçabilité complète de la chaîne de valeur.

PARTIE 5: DEMO

1) Tableau de bord:

Le tableau de bord offre une vue de synthèse de l'exploitation pour un mois donné. Il regroupe en un seul écran les indicateurs clés : effectif du cheptel, montant des ventes, total des dépenses et mises bas à venir. Il joue le rôle de poste de pilotage : d'un coup d'œil, l'éleveur mesure l'activité et repère les points nécessitant une attention immédiate.



2) Reproduction : groupe et alerte de mise bas

Ce module gère le cycle de reproduction par groupe : saillie, gestation et mise bas. À partir de la date de saillie, l'application calcule automatiquement la date prévue de mise bas (durée de gestation de 114 jours) et surveille son échéance. Lorsqu'une mise bas approche, dans les cinq jours, le système génère automatiquement une alerte. Il s'agit de la fonctionnalité métier la plus avancée de l'application, reposant sur des règles de gestion et une surveillance automatisée.

MADAPORC

Version 4.0.0

Tableau de bord

Calendrier

Mouvements

CHEFTEL

Lots de porcs

REPRODUCTION

GROUPE

Analyse reproductrice

Alertes

SANTÉ

Vaccins

Vaccinations

Suivi sanitaire

COMMERCE

Clients

Ventes

STOCKS & FINANCE

Ingrédients

Mouvements stock

Dépenses

DONNÉES

Import / Export

Rapports

ADMINISTRATION

Déconnexion

Reproduction / Groupes / GRP-1784122736020

Administrateur

GRP-1784122736020

Lot mère - Aucun lot mère - 5 femelles

Voir l'analyse

Confirmer la mise bas

Évolution du cycle

Jours restants avant mise bas : 114

0%

Saillie : 2026-07-24

Prévue : 2026-11-15

Détails du groupe

Code groupe	GRP-1784122736020
Femelles concernées	5
Mères utilisées	1
Date de saillie	2026-07-24
Duration gestation	114 jours
Mise bas prévue	2026-11-15
Statut	SAILLIE
Observation	Aucune observation

Résultat de mise bas

La mise bas n'est pas encore confirmée.

Confirmer la mise bas

À 2026 MADAPORC

MADAPORC

Version 4.0.0

Tableau de bord

Calendrier

Mouvements

CHEFTEL

Lots de porcs

REPRODUCTION

GROUPE

Analyse reproductrice

Alertes

SANTÉ

Vaccins

Vaccinations

Suivi sanitaire

COMMERCE

Clients

Ventes

STOCKS & FINANCE

Ingrédients

Mouvements stock

Dépenses

DONNÉES

Import / Export

Rapports

ADMINISTRATION

Déconnexion

Reproduction / Groupes / GRP-1784122788487

Administrateur

GRP-1784122788487

Lot mère - Aucun lot mère - 2 femelles

Voir l'analyse

Confirmer la mise bas

Évolution du cycle

Jours restants avant mise bas : 0

100%

Saillie : 2026-03-23

Prévue : 2026-07-15

Détails du groupe

Code groupe	GRP-1784122788487
Femelles concernées	2
Mères utilisées	1
Date de saillie	2026-03-23
Duration gestation	114 jours
Mise bas prévue	2026-07-15
Statut	SAILLIE
Observation	Aucune observation

Résultat de mise bas

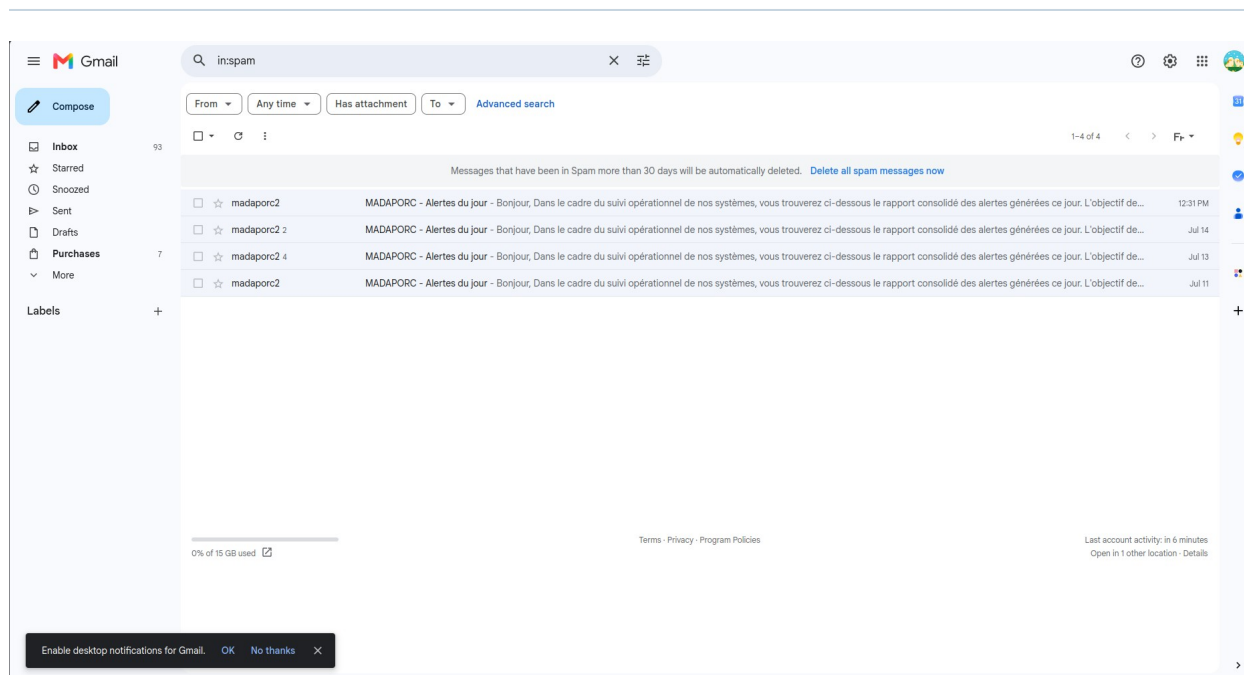
La mise bas n'est pas encore confirmée.

Confirmer la mise bas

À 2026 MADAPORC

3) Alerte de reproduction + email:

En complément des alertes affichées dans l'interface, l'application envoie automatiquement un e-mail récapitulatif à l'éleveur lorsqu'une mise bas est imminente. Cet envoi est déclenché par une tâche planifiée qui s'exécute en arrière-plan, sans intervention de l'utilisateur. Ce mécanisme illustre l'automatisation du suivi et évite à l'éleveur de devoir consulter l'application en permanence.



4) Détail d'un lot et suivi d'un pesée:

La fiche détaillée d'un lot centralise toutes les informations le concernant : composition, historique des mouvements (entrées, décès, transferts) et suivi des pesées. L'enregistrement régulier du poids moyen permet de suivre la croissance du lot dans le temps et d'évaluer les performances d'engraissement. C'est le cœur du suivi technique de l'élevage.

MADAPORC
Général d'élevage

Chapitre / Lots / LOT-F-002

FICHE LOT PORCIN
LOT-F-002 Femelle Actif
Large White - REPRODUCTION - Origine : ACHAT

EFFECTIF ACTUEL
10
Initial : 10

SEXE DU LOT
FEMELLE
Lot non mixte

RACE
Large White
Classification génétique

STATUT
ACTIF
État opérationnel

Informations du lot
Résumé technique et zootechnique du lot.

Code lot LOT-F-002	Date de création 2026-03-15
Sexe FEMELLE	Race Large White
Objectif REPRODUCTION	Origine ACHAT
Effectif initial 10	Effectif actuel 10
Lot parent ---	Groupe reproduction origine ---

Description
Aucune description renseignée pour ce lot.

Actions rapides
Gestion opérationnelle du lot.

- Suivre les pesées**
Historique du poids moyen.
- Voir les mouvements**
Entrées, sorties et ajournements.
- Modifier le lot**
Mettre à jour les informations.

Traçabilité
Informations système.

Créé le
2026-07-15T16:37:49.241446

Dernière modification
2026-07-15T16:37:49.300091

À 2026 MADAPORC

MADAPORC

Tableau de bord

Calendrier

Mouvements

CHEPTEL

Lots de porcs

REPRODUCTION

Groupes

Analyse reproductive

Alertes

SANTÉ

Vaccins

Vaccinations

Suivi sanitaires

COMMERCE

Clients

Ventes

STOCKS & FINANCE

Ingrédients

Mouvements stock

Dépenses

DONNÉES

Import / Export

Rapports

ADMINISTRATION

Déconnexion

Cheptel / Lots / LOT-F-002 / Pesées

Administrateur

Retour au lot

Suivi du poids

Lot LOT-F-002

Historique des pesées

DATE	POIDS MOYEN (kg)	OBSERVATION
2026-03-15	100.00	Poids de départ à la création du lot.

Nouvelle pesée

Poids moyen (kg) *

Doit être supérieur à 0.

Date de pesée *

jj / mm / aaaa

Ne peut pas être dans le futur.

Observation

Enregistrer la pesée

Â© 2026 MADAPORC

5) Vente:

Ce module assure la gestion commerciale de l'exploitation. Chaque vente est enregistrée avec son client, sa date et le détail des lots vendus (quantité, poids, prix), le montant total étant calculé automatiquement. La vue de détail retrace l'ensemble de la transaction et relie directement la vente aux lots concernés, assurant la traçabilité entre le cheptel et les recettes.

MADAPORC

Tableau de bord

Calendrier

Mouvements

CHEPTEL

Lots de porcs

REPRODUCTION

Groupes

Analyse reproductive

Alertes

SANTÉ

Vaccins

Vaccinations

Suivi sanitaires

COMMERCE

Clients

Ventes

STOCKS & FINANCE

Ingrédients

Mouvements stock

Dépenses

DONNÉES

Import / Export

Rapports

ADMINISTRATION

Déconnexion

Commerce / Ventes / VTE-00001

Administrateur

Reçu PDF

Annuler

VTE-00001

Valable

Client : Bouchons Centrale - 2026-07-15

Détail des lignes

LOT	QUANTITÉ	PRIX UNITAIRE (AR)	TOTAL (AR)
LOT-F-002	2	1 000 000	2 000 000
Montant total			2 000 000 Ar

Â© 2026 MADAPORC

MADAPORC

Recu de vente

VTE-00001

Client : Boucherie Centrale

Date : 2026-07-15

Statut : VALIDEE

Lot	Quantite	Prix unitaire	Montant
LOT-F-002	2	1 000 000 Ar	2 000 000 Ar
Montant total			2 000 000 Ar

6) Rapport et suivi financier:

Cette page synthétise l'activité de l'exploitation sous forme de rapports exploitables : suivi des dépenses par catégorie, recettes issues des ventes et indicateurs de production. Elle permet à l'éleveur de disposer d'une vision consolidée pour analyser la rentabilité et appuyer ses décisions de gestion.

MADAPORC

Élevage et élevage

Tableau de bord

Calendrier

Mouvements

CHEPTTEL

Lots de porcs

REPRODUCTION

Groupes

Analyses reproductives

Alertes

SANTÉ

Vaccins

Vaccinations

Suivi sanitaire

COMMERCE

Clients

Ventes

STOCKS & FINANCE

Ingédients

Mouvements stock

Dépenses

DONNEES

Import / Export

Rapports

ADMINISTRATION

Déconnexion

Données / Rapports

Administrateur ADMIN

Rapports

Générez des rapports PDF par domaine, ou exportez les données brutes

Bilan financier

Date début: g / mm / aaaa

Date fin: g / mm / aaaa

Calculer

Total ventes (validées)

2 000 000 Ar

(4 ventes)

Total dépenses

200 000 Ar

(2 dépenses)

Bénéfice net

1 800 000 Ar

Tout le groupe

Sanitaire

Vaccinations et suivis sur la période.

Générer le PDF

Commercial

Ventes, clients et chiffres d'affaires.

Générer le PDF

Financier

Recettes, dépenses et bénéfice net.

Générer le PDF

Reproduction

Groupes, mises bas et taux de fertilité.

Générer le PDF

Export global Excel

Toutes les données dans un classeur.

Télécharger Excel

Import / Export

Outils détaillés par module.

Ouvrir les outils

Â© 2026 MADAPORC

INDICATEURS OPÉRATIONNELS

Finances Reproduction Cheptel

Pilotage financier mensuel (Ar)



Conclusion:

Le projet MADAPORC répond à un besoin concret et pressant du secteur porcin à Madagascar : remplacer une gestion artisanale, source d'erreurs et de pertes de productivité, par un outil numérique fiable, structuré et évolutif. En s'appuyant sur une architecture robuste (Java/Spring Boot, PostgreSQL) et une conception modulaire centrée sur les règles métier, l'application pose les bases d'une gestion technique rigoureuse des lots, de la reproduction, de la santé et des flux financiers.

Au-delà de la simple digitalisation, MADAPORC se veut un véritable levier d'aide à la décision, grâce à des indicateurs automatisés et une traçabilité complète. Les perspectives d'évolution — API REST, intelligence artificielle prédictive, intégration géospatiale — ouvrent la voie à un outil durable, capable de s'adapter aux mutations du secteur et d'accompagner la montée en compétences des éleveurs et des exploitants.

Ainsi, MADAPORC ne se limite pas à une application de gestion : il incarne une vision moderne, intégrée et responsable de l'élevage porcin à Madagascar.

Synthèse de présence de chaque membre tout au long du projet:

Membre	Nb absence	Date absence	Cause	Point
Tsanta	2	02/07/2026 et 01/07/2026	Malade	1
Faniry	Jamais présent	0	Injustifié	-3
Fanilo	0	0	Toujours présent	1
Noah	0	0	Toujours présent	1
Sarobidy	2	20/06/2026	Injustifié	1
Davida	1	13/06/2026	Malade	1
Mandresy	1	04/07/2026	Toujours présent	1
Manoa	0	0	Toujours présent	1
Irina	Jamais présent	0	Injustifié	-3